

Tool Learning and Function Calling for LLM Agents

Abstract

Tool learning and function calling have become central to the transition from large language models from static text generators into interactive agents that can retrieve information, invoke software, operate web interfaces, manipulate devices, and act in external environments. The area brings together several previously distinct research traditions: semantic parsing, task-oriented dialogue, program synthesis, neuro-symbolic reasoning, embodied planning, retrieval augmentation, and more recent agentic prompting and post-training for LLMs. In contemporary systems, “tool learning” usually refers to the broader problem of deciding whether tools are needed, selecting or retrieving them, generating well-formed calls, integrating observations into subsequent reasoning, and terminating with a grounded response or further action. “Function calling” is the more schema-constrained subproblem in which the model must output a structured invocation—often JSON-like or AST-like—whose arguments are executable by downstream software. This survey synthesizes the literature around these intertwined problems, emphasizing method families, benchmark design, safety, evaluation, and open research questions. [1–14, 15–31, 61–98, 138–148]. ¹

The modern literature has converged on a useful conceptual distinction among at least three broad paradigms. First, prompting-centric approaches elicit tool use at inference time through few-shot in-context examples, interleaved reasoning-action traces, or planner-executor decompositions. Second, supervised tool-learning approaches internalize function calling and tool-selection behaviors through synthetic or curated training data, increasingly at scale and often with thousands of tools or APIs. Third, post-training and reinforcement-learning approaches optimize strategic tool use, tool abstention, search behavior, and long-horizon execution quality. Running across these paradigms is a rapidly expanding benchmark ecosystem that now measures not only one-shot schema validity but also multi-turn interaction, compositionality, uncertainty handling, stateful execution, robustness to tool perturbations, and resistance to prompt injection or malicious tool outputs. [27–39, 61–73, 74–98, 99–131]. ²

Introduction

The intellectual roots of tool learning for LLM agents predate contemporary LLMs by nearly two decades. Early semantic parsing research treated language understanding as the problem of translating utterances into executable meaning representations over databases or symbolic programs, while task-oriented dialogue research focused on eliciting arguments, deciding when to issue API calls, and grounding responses in structured backends. These lines of work established several enduring design principles: the value of typed structures, intermediate programs, executable supervision, and environment-grounded evaluation. Modern function calling inherits all of these concerns, but now places them inside a much more general generative model that must flexibly handle open-domain language, unseen tools, and long-horizon interaction. [1–14]. ³

The immediate prehistory of the field is the 2022–2023 wave of reasoning and program-aided prompting. Chain-of-thought prompting, self-consistency, zero-shot reasoning, least-to-most prompting, tree/graph search, and program-of-thought methods demonstrated that LLM performance could improve

dramatically when reasoning was externalized into intermediate textual or executable traces. In parallel, Binder, TALM, PAL, and related work showed that external programs and tools could serve as structured interfaces for reasoning rather than mere post hoc utilities. This period shifted the question from whether language models could use tools to how tool invocation should be represented, selected, supervised, and evaluated. [15–24]. ⁴

A second acceleration came from prompt-based agents and orchestration frameworks such as ReAct, Reflexion, Self-Refine, CRITIC, ART, Chameleon, HuggingGPT, TaskMatrix.AI, OpenAGI, ReWOO, and LLM-Compiler. These systems reframed the LLM as a policy over external actions: searching, calling APIs, executing code, delegating to models, or navigating an environment. The field simultaneously expanded into embodied settings, with SayCan, Inner Monologue, Code as Policies, Voyager, and later computer-use agents showing that tool use is not limited to text APIs but generalizes to robotic skills, software interfaces, and reusable executable skills. [25–60]. ⁵

What distinguishes the current phase is the emergence of function calling as a first-class target for training and evaluation. Toolformer introduced self-supervised API annotation; Gorilla, ToolLLM, API-Bank, ToolAlpaca, GPT4Tools, and successive systems showed that large-scale instruction tuning over real or synthetic tool corpora can produce models that are substantially better at schema grounding and argument completion; and newer work has expanded the problem to open-world retrieval, multi-turn composition, and realistic benchmark settings such as BFCL, ToolSandbox, ToolTalk, τ -bench, ACEBench, and Live API-Bench. At the same time, safety work such as ToolEmu, AgentDojo, ToolSword, and prompt-injection studies has made clear that tool access magnifies both capability and risk. [61–131]. ⁶

This survey adopts a broad but agent-centered scope. We treat function calling as one operational substrate for tool use and study it alongside related tool-learning questions: when to call, what to call, how to parameterize calls, how to integrate observations, how to plan across calls, how to abstain or seek clarification, and how to remain robust and safe in the presence of ambiguity, tool evolution, and adversarial interference. We emphasize methodological continuity across these subproblems rather than isolating “function calling” as a narrow formatting task. [61–98, 138–148]. ⁷

Methods

A useful organizing principle is the pipeline from user intent to external action: the model must interpret the task, decide whether tool use is warranted, retrieve or narrow the tool set, produce executable calls, integrate the returned state, and possibly repeat the cycle. Existing work differs mainly in where learning occurs, what supervision is available, and how much of the procedure is explicit versus internalized.

Prompt-based and test-time tool orchestration. The earliest LLM-agent tool systems largely relied on prompting rather than parameter updates. ReAct interleaved reasoning and acting; Reflexion and Self-Refine introduced self-feedback loops; CRITIC used tool-interactive verification; ART selected multi-step reasoning-and-tool traces from a task library; and Chameleon, HuggingGPT, OpenAGI, Visual ChatGPT, and TaskMatrix.AI treated the LLM as a controller over heterogeneous expert modules. ReWOO and LLM-Compiler moved further by decoupling planning from execution and by exposing opportunities for parallel or partially ordered function calling. This family remains attractive because it is modular, model-agnostic, and easy to extend to new tools, but it pays for this flexibility with prompt length, latency, instability over long trajectories, and sensitivity to tool descriptions. [21–39, 45–60]. ⁸

Within this family, a recurring idea is that code and programs are unusually effective mediators between language and action. PAL and Program-of-Thoughts externalized computation; Binder translated hybrid symbolic-neural tasks into executable programs; ViperGPT used Python execution for multimodal

reasoning; Code as Policies compiled policies from language into embodied-control code; and LATM reframed agents as both tool users and prospective tool makers. The significance of these works for function calling is conceptual: they imply that “a tool call” is best viewed not as a formatting artifact but as a typed program fragment with compositional semantics and execution consequences. [21–24, 40–42]. ⁹

Supervised internalization of tool use. Toolformer marked a turning point by showing that a language model can label its own corpus with API calls and keep only calls that improve next-token prediction. Although the setup involved a small tool set, the broader contribution was methodological: tool use became a learnable latent intervention in standard language model training. Later work broadened supervision through synthetic instruction data and larger tool inventories. ToolAlpaca and GPT4Tools used strong-model-generated supervision; ToolLLM and API-Bank scaled to thousands of APIs and more diverse stages of the pipeline; Gorilla combined fine-tuning with document retrieval to support changing APIs; and Granite-Function Calling, xLAM, and related models focused more explicitly on schema fidelity, argument extraction, and relevance detection in practical function-calling settings. [61–73]. ¹⁰

One of the most important design shifts in supervised tool learning is the move from closed-world invocation to open-world retrieval. Tool documentation alone can support surprisingly strong zero-shot behavior, but as tool libraries scale, the model must first identify a tractable subset of candidate tools. AnyTool addresses this with hierarchical retrieval; Meta-Tool turns open-world function calling into a hypothesize–retrieve–invoke pipeline; EASYTOOL compresses verbose tool specs into more token-efficient instructions; MCP-Zero proactively retrieves toolchains instead of stuffing thousands of schemas into the prompt; and recent dependency-aware retrieval methods model relations among tools rather than ranking them independently. This line of work effectively merges function calling with retrieval-augmented generation, but the retrieved objects are executable affordances rather than factual documents. [62, 67, 70, 79–82, 103–108]. ¹¹

Another supervised trend is process-oriented tuning. T-Eval exposed the value of evaluating invocation step by step rather than only at the final answer; Agent-FLAN mixed positive and negative trajectories for better agent tuning; ToolAlign explicitly targeted alignment between intent, tool use, and human preferences; and ToRA demonstrated that rationale-augmented training can substantially help when tools are invoked for mathematical reasoning. More recent work, including BUTTON, ToolACE, APIGen, and FunReason, emphasizes synthetic data generation that covers multi-turn interaction, nested calls, ambiguous instructions, and verifiable call correctness. A common theme is that data quality—especially correctness of arguments and trajectory structure—matters at least as much as sheer data scale. [74–78, 99–102, 110–111]. ¹²

Evaluation-driven function calling research. A notable feature of this area is that benchmarks have not merely measured progress; they have defined the subproblems that matter. API-Bank and ToolLLM/StableToolBench foregrounded tool retrieval and API-scale generalization. BFCL established schema-aware function-calling evaluation as a standalone benchmarking target and pushed the community toward AST-style or rule-based exactness rather than free-form textual judgments. ToolEyes and HammerBench decomposed errors into fine-grained categories such as intent mismatch, parameter-name confusion, or multi-turn slot drift. NESTFUL highlighted nested and compositional call chains; When2Call and WTU-EVAL put abstention and non-use decisions on equal footing with invocation; ToolTalk and ToolSandbox stressed stateful, conversational interaction; τ -bench and ACEBench emphasized user-agent-tool interaction under policies; and the latest WildToolBench, MCPVerse, and Live API-Bench scale the problem toward more realistic behavior and larger executable tool spaces. [65–66, 74, 79–93]. ¹³

These benchmarks make visible a crucial distinction between local correctness and global task success. A model may choose the right function schema yet fail because it should have asked a clarification question; it may return valid JSON but select the wrong tool; it may complete each step correctly but violate a domain policy; or it may succeed once but behave inconsistently across repeated trials. This is why recent evaluation increasingly reports step-level metrics, pass@k or pass^k consistency measures, dialogue-level success, policy adherence, and utility-under-attack rather than only one-shot exact match. [75, 81–90, 95–98]. ¹⁴

Reinforcement learning, search, and post-training. A newer family of methods treats tool use as sequential decision making rather than static structured prediction. ReTool optimizes strategic tool-use behavior with reinforcement learning; reinforcement-learning methods for agent planning and search-oriented post-training aim to encourage models to retrieve, verify, or branch more effectively; DeepResearcher extends this into real-world environments; and asynchronous or planner-decoupled function-calling systems optimize latency and throughput in addition to call quality. The deeper idea is that function calling quality depends on policies over tools, not only on correct argument filling. Reward design therefore increasingly includes task completion, invocation efficiency, uncertainty handling, and sometimes process supervision. [117–123]. ¹⁵

A practical consequence of this shift is the growing separation between capability acquisition and inference-time control. Some systems rely on heavily fine-tuned small or medium models for low-latency function calling on edge devices; others keep larger frontier models frozen and improve them via retrieved tool context or search-based deliberation. HAMMER, TinyAgent, and related work suggest that constrained function-calling behavior can be learned efficiently on smaller models when the action space and masking strategy are carefully designed, while xLAM and open-world retrieval systems suggest that large action spaces require better representations of tool libraries rather than just larger decoders. [71–73, 80, 103–110, 122–123]. ¹⁶

Multimodal, embodied, and domain-specific tool use. Function calling in text APIs is only one slice of the broader tool-learning problem. In robotics and embodied control, tools are skills or policies whose applicability depends on affordances and environment state. In GUI and computer-use settings, the “function” is effectively a high-level action grounded in pixels, UI parsing, and latent application state. In domain-specific scientific settings, such as chemistry, data analysis, or coding, tools often supply trustworthy computation, search, or symbolic reasoning unavailable to the base model. These settings are important because they stress different weaknesses: perception and grounding for GUI tools, spatiotemporal feedback in robotics, and semantic precision for scientific or coding tools. [25–26, 40–60, 132–139]. ¹⁷

Across all these method families, one empirical regularity stands out: tool use is rarely bottlenecked by surface syntax alone. Even when models produce executable calls, they often fail in deciding whether a tool is needed, identifying missing information, maintaining state across turns, coping with paraphrased or degraded tool descriptions, composing multiple tools, or safely interpreting untrusted tool outputs. The strongest recent papers consequently target distributions of interactional uncertainty rather than just stronger next-token prediction on clean single-turn traces. [78–98, 124–131]. ¹⁸

Challenges and Prospects

The first major challenge is **semantic grounding under open-world tool growth**. Tool libraries are expanding faster than any static fine-tuning dataset can cover, and real deployments increasingly involve thousands of heterogeneous schemas, evolving APIs, nested dependencies, language variability, and hidden business rules. This suggests that future systems will need hybrid parametric–nonparametric

architectures in which the model internalizes generic invocation priors while retrieving fresh tool affordances and dependency structure at inference time. The most promising directions here combine hierarchical retrieval, concise tool representations, dependency-aware retrieval, proactive tool discovery, and planner-executor structures that make intermediate state explicit. [67, 70, 80, 91–93, 99–108]. ¹⁹

The second challenge is **decision quality beyond schema validity**. Benchmarks such as When2Call, WTU-EVAL, τ -bench, ToolSandbox, and ACEBench show that a useful agent must know when not to call tools, when to ask for clarification, when to defer, and how to satisfy policy constraints over multi-turn interaction. This is a substantially harder problem than producing well-formed JSON. It pushes the field toward richer notions of uncertainty calibration, active information gathering, refusal and abstention policies, and policy-aware dialogue management. In other words, the frontier is moving from function formatting toward agentic judgment. [81–90]. ²⁰

The third challenge is **robustness and security**. Tool access enlarges the attack surface because tool outputs, retrieved documents, or malicious schemas can inject instructions that compete with the user’s objective. ToolSword, ToolEmu, AgentDojo, prompt-injection work on tool selection, Impropter, PromptArmor, and AgentSentry collectively show that even strong models can be induced to misuse tools, leak information, or deviate from task goals. Importantly, this is not merely a red-teaming afterthought: tool learning structurally entangles data and control, so future research must integrate defenses at the model, retrieval, execution, and protocol layers. Sandboxed execution, provenance-aware policies, sanitizer or firewall modules, selective quitting, and auditable planners are likely to become standard components rather than optional extras. [95–97, 124–131]. ²¹

A final challenge is **standardization without overfitting to narrow benchmarks**. The field currently risks splitting into partially incompatible subcommunities: API-based function calling, web agents, GUI agents, embodied planners, and protocol-driven tool ecosystems. Yet the emerging literature also suggests convergence. Real-world benchmarks are expanding toward larger, executable action spaces; prompted and trained methods are converging on similar tool-retrieval abstractions; and survey work increasingly frames tool use, planning, and memory as interacting components of a common agent architecture. A compelling prospect is therefore a unified theory of externalized cognition for LLM agents, in which tools, memories, skills, and environments are all retrieved and manipulated through typed interfaces with shared evaluation principles. [91–93, 138–148]. ²²

Conclusion

Tool learning and function calling are no longer peripheral augmentations to LLMs; they are rapidly becoming the operational core of agentic language systems. The literature has progressed from semantic parsing and dialogue-era API grounding to prompt-based orchestration, self-supervised and synthetic supervision at large scale, open-world tool retrieval, stateful and policy-aware evaluation, and reinforcement-learning-based optimization. The main lesson of the surveyed work is that successful tool use is an end-to-end competence, not a single decoding trick: it involves judgment, retrieval, planning, abstention, robustness, and safe execution in addition to argument formatting. Progress will therefore depend less on one dominant paradigm than on principled combinations of retrieval, structured generation, post-training, and secure runtime design. If the field can align these components with realistic evaluation and robust safety practices, tool-using LLM agents may become one of the most important interfaces between foundation models and the external world. [1–148]. ²³

References

- [1] L. S. Zettlemoyer and M. Collins, “Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorical Grammars,” *Proceedings of UAI*, 2005.
- [2] Y. W. Wong and R. J. Mooney, “Learning Synchronous Grammars for Semantic Parsing with Lambda Calculus,” *Proceedings of ACL*, 2007.
- [3] P. Liang, M. I. Jordan, and D. Klein, “Learning Dependency-Based Compositional Semantics,” *Proceedings of ACL*, 2011.
- [4] J. Berant, A. Chou, R. Frostig, and P. Liang, “Semantic Parsing on Freebase from Question-Answer Pairs,” *Proceedings of EMNLP*, 2013.
- [5] L. Dong and M. Lapata, “Language to Logical Form with Neural Attention,” *Proceedings of ACL*, 2016.
- [6] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning,” *arXiv:1709.00103*, 2017.
- [7] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, et al., “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” *Proceedings of EMNLP*, 2018, *arXiv:1809.08887*.
- [8] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models,” *Proceedings of EMNLP*, 2021, *arXiv:2109.05093*.
- [9] A. Bordes, Y.-L. Boureau, and J. Weston, “Learning End-to-End Goal-Oriented Dialog,” *arXiv:1605.07683*, 2016.
- [10] M. Eric, L. Krishnan, F. Charette, and C. D. Manning, “Key-Value Retrieval Networks for Task-Oriented Dialogue,” *Proceedings of SIGDIAL*, 2017, *arXiv:1705.05414*.
- [11] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić, “MultiWOZ: A Large-Scale Multi-Domain Wizard-of-Oz Dataset for Task-Oriented Dialogue Modelling,” *Proceedings of EMNLP*, 2018, *arXiv:1810.00278*.
- [12] A. Rastogi, X. Zang, S. Sunkara, R. Gupta, and P. Khaitan, “Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset,” *Proceedings of AAAI*, 2020, *arXiv:1909.05855*.
- [13] M. Heck, C. van Niekerk, N. Lubis, C. Geishauser, H.-C. Lin, M. Moresi, and M. Gašić, “TripPy: A Triple Copy Strategy for Value Independent Neural Dialog State Tracking,” *Proceedings of SIGDIAL*, 2020, *arXiv:2005.02877*.
- [14] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. Routledge, and W. Y. Wang, “FinQA: A Dataset of Numerical Reasoning over Financial Data,” *Proceedings of EMNLP*, 2021, *arXiv:2109.00122*.

- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Proceedings of NeurIPS*, 2022, arXiv:2201.11903.
- [16] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” arXiv:2203.11171, 2022.
- [17] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large Language Models are Zero-Shot Reasoners,” *Proceedings of NeurIPS*, 2022, arXiv:2205.11916.
- [18] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models,” *Proceedings of ICLR*, 2023, arXiv:2205.10625.
- [19] S. Yao, D. Yu, J. Zhao, I. Shafran, K. Narasimhan, and Y. Cao, “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *Proceedings of NeurIPS*, 2023, arXiv:2305.10601.
- [20] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoeffler, “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” *Proceedings of AAAI*, 2024, arXiv:2308.09687.
- [21] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “PAL: Program-Aided Language Models,” *Proceedings of ICML*, 2023, arXiv:2211.10435.
- [22] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks,” *Transactions on Machine Learning Research*, 2023, arXiv:2211.12588.
- [23] Z. Cheng, T. Xie, P. Shi, C. Li, R. Nadkarni, Y. Hu, C. Xiong, D. Radev, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and T. Yu, “Binding Language Models in Symbolic Languages,” *Proceedings of ICLR*, 2023, arXiv:2210.02875.
- [24] A. Parisi, Y. Zhao, and N. Fiedel, “TALM: Tool Augmented Language Models,” arXiv:2205.12255, 2022.
- [25] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, C. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” *Proceedings of CoRL*, 2023, arXiv:2204.01691.
- [26] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al., “Inner Monologue: Embodied Reasoning through Planning with Language Models,” *Proceedings of CoRL*, 2023, arXiv:2207.05608.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *Proceedings of ICLR*, 2023, arXiv:2210.03629.
- [28] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” arXiv:2303.11366, 2023.

- [29] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, et al., “Self-Refine: Iterative Refinement with Self-Feedback,” Proceedings of NeurIPS, 2023, arXiv:2303.17651.
- [30] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, “CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing,” Proceedings of ICLR, 2024, arXiv:2305.11738.
- [31] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, and M. T. Ribeiro, “ART: Automatic Multi-Step Reasoning and Tool-Use for Large Language Models,” arXiv:2303.09014, 2023.
- [32] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao, “Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models,” arXiv:2304.09842, 2023.
- [33] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face,” arXiv:2303.17580, 2023.
- [34] Y. Liang, C. Wu, T. Song, W. Wu, Y. Xia, Y. Liu, Y. Ou, S. Lu, L. Ji, S. Mao, et al., “TaskMatrix.AI: Completing Tasks by Connecting Foundation Models with Millions of APIs,” Intelligent Computing, 2024, arXiv:2303.16434.
- [35] Y. Ge, W. Hua, K. Mei, J. Ji, J. Tan, S. Xu, Z. Li, and Y. Zhang, “OpenAGI: When LLM Meets Domain Experts,” NeurIPS Datasets and Benchmarks Track, 2023, arXiv:2304.04370.
- [36] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, “Visual ChatGPT: Talking, Drawing and Editing with Visual Foundation Models,” arXiv:2303.04671, 2023.
- [37] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, “ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models,” arXiv:2305.18323, 2023.
- [38] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, “An LLM Compiler for Parallel Function Calling,” Proceedings of ICML, 2024, arXiv:2312.04511.
- [39] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An Open-Ended Embodied Agent with Large Language Models,” Proceedings of NeurIPS, 2023, arXiv:2305.16291.
- [40] D. Surís, S. Menon, and C. Vondrick, “ViperGPT: Visual Inference via Python Execution for Reasoning,” Proceedings of ICCV, 2023, arXiv:2303.08128.
- [41] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as Policies: Language Model Programs for Embodied Control,” arXiv:2209.07753, 2022.
- [42] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” Proceedings of ICLR, 2024, arXiv:2305.17126.
- [43] J. S. Park, J. O’ Brien, C. Cai, M. Morris, P. Liang, and M. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” Proceedings of UIST, 2023, arXiv:2304.03442.
- [44] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023.

- [45] S. Sun, Y. Liu, S. Wang, D. Iter, C. Zhu, and M. Iyyer, “PEARL: Prompting Large Language Models to Plan and Execute Actions Over Long Documents,” *Proceedings of EACL*, 2024.
- [46] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, H. Huang, C. Li, K. Wang, R. Yao, Y. Tian, and S. Li, “RestGPT: Connecting Large Language Models with Real-World Applications via RESTful APIs,” *arXiv:2306.06624*, 2023.
- [47] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a Generalist Agent for the Web,” *Proceedings of NeurIPS*, 2023, *arXiv:2306.06070*.
- [48] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models,” *Proceedings of ACL*, 2024, *arXiv:2401.13919*.
- [49] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A Realistic Web Environment for Building Autonomous Agents,” *Proceedings of ICLR*, 2024, *arXiv:2307.13854*.
- [50] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, “VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks,” *Proceedings of ACL*, 2024, *arXiv:2401.13649*.
- [51] A. Drouin, M. Gasse, M. Caccia, I. Laradji, M. Del Verme, T. Marty, L. Boisvert, M. Thakkar, Q. Cappart, D. Vazquez, N. Chapados, and A. Lacoste, “WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks?,” *Proceedings of ICML*, 2024, *arXiv:2403.07718*.
- [52] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents,” *Proceedings of ACL*, 2024, *arXiv:2405.15778*.
- [53] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al., “AgentBench: Evaluating LLMs as Agents,” *Proceedings of ICLR*, 2024, *arXiv:2308.03688*.
- [54] Y. Li, S. Men, B. Chen, R. Wang, Y. Qiu, Y. Zhang, H. Ma, et al., “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments,” *arXiv:2404.07972*, 2024.
- [55] W. Tan, W. Zhang, X. Xu, H. Xia, Z. Ding, B. Li, B. Zhou, J. Yue, J. Jiang, Y. Li, et al., “Cradle: Empowering Foundation Agents towards General Computer Control,” *Proceedings of ICML*, 2025, *arXiv:2403.03186*.
- [56] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, “Agent S: An Open Agentic Framework that Uses Computers Like a Human,” *Proceedings of ICLR*, 2025, *arXiv:2410.08164*.
- [57] L. Zheng, R. Wang, X. Wang, and B. An, “Synapse: Trajectory-as-Exemplar Prompting with Memory for Computer Control,” *Proceedings of ICLR*, 2024, *arXiv:2306.07863*.
- [58] K. Q. Lin, L. Li, D. Gao, Z. Yang, S. Wu, Z. Bai, W. Lei, L. Wang, and M. Z. Shou, “ShowUI: One Vision-Language-Action Model for GUI Visual Agent,” *Proceedings of CVPR*, 2025.
- [59] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, “OmniParser for Pure Vision Based GUI Agent,” *arXiv:2408.00203*, 2024.

- [60] J. Yang, R. Tan, Q. Wu, R. Zheng, B. Peng, Y. Liang, Y. Gu, M. Cai, S. Ye, J. Jang, et al., “Magma: A Foundation Model for Multimodal AI Agents,” *Proceedings of CVPR*, 2025.
- [61] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *Proceedings of NeurIPS*, 2023, arXiv:2302.04761.
- [62] C.-Y. Hsieh, S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, and T. Pfister, “Tool Documentation Enables Zero-Shot Tool-Usage with Large Language Models,” arXiv:2308.00675, 2023.
- [63] Q. Tang, Z. Deng, H. Lin, X. Han, Q. Liang, B. Cao, and L. Sun, “ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases,” arXiv:2306.05301, 2023.
- [64] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, “GPT4Tools: Teaching Large Language Model to Use Tools via Self-Instruction,” arXiv:2305.18752, 2023.
- [65] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al., “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs,” arXiv:2307.16789, 2023.
- [66] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs,” *Proceedings of EMNLP*, 2023, arXiv:2304.08244.
- [67] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” *Proceedings of NeurIPS*, 2024, arXiv:2305.15334.
- [68] V. K. Srinivasan, Z. Dong, B. Zhu, B. Yu, H. Mao, D. Mosk-Aoyama, K. Keutzer, J. Jiao, and J. Zhang, “NexusRaven: A Commercially-Permissive Language Model for Function Calling,” *NeurIPS Workshop on Foundation Models for Decision Making*, 2023.
- [69] S. Hao, T. Liu, Z. Wang, and Z. Hu, “ToolkenGPT: Augmenting Frozen Language Models with Massive Tools via Tool Embeddings,” arXiv:2305.11554, 2023.
- [70] Y. Du, F. Wei, and H. Zhang, “AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls,” arXiv:2402.04253, 2024.
- [71] W. Chen and Z. Li, “Octopus v2: On-Device Language Model for Super Agent,” arXiv:2404.01744, 2024.
- [72] I. Abdelaziz, K. Basu, M. Agarwal, S. Kumaravel, M. Stallone, R. Panda, Y. Rizk, G. P. S. Bhargav, M. Crouse, C. Gunasekara, et al., “Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-Task Learning of Granular Tasks,” *Proceedings of EMNLP Industry Track*, 2024.
- [73] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng, et al., “xLAM: A Family of Large Action Models to Empower AI Agent Systems,” *Proceedings of NAACL*, 2025.
- [74] Z. Guo, S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li, Z. Liu, M. Sun, and Y. Liu, “StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models,” *Findings of ACL*, 2024.

- [75] Z. Chen, W. Du, W. Zhang, K. Liu, J. Liu, M. Zheng, J. Zhuo, S. Zhang, D. Lin, K. Chen, and F. Zhao, “T-Eval: Evaluating the Tool Utilization Capability of Large Language Models Step by Step,” Proceedings of ACL, 2024.
- [76] Z. Chen, K. Liu, Q. Wang, W. Zhang, J. Liu, D. Lin, K. Chen, and F. Zhao, “Agent-FLAN: Designing Data and Methods of Effective Agent Tuning for Large Language Models,” Findings of ACL, 2024.
- [77] Z. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, and Y. Lin, “Towards Tool Use Alignment of Large Language Models,” Proceedings of EMNLP, 2024.
- [78] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, “ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving,” Proceedings of ICLR, 2024, arXiv:2309.17452.
- [79] Y. Huang, J. Shi, Y. Li, C. Fan, S. Wu, Q. Zhang, Y. Liu, P. Zhou, Y. Wan, N. Z. Gong, and L. Sun, “MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use,” arXiv:2310.03128, 2023.
- [80] S. Qin, Y. Zhu, L. Mu, S. Zhang, and X. Zhang, “Meta-Tool: Unleash Open-World Function Calling Capabilities of General-Purpose Large Language Models,” Proceedings of ACL, 2025.
- [81] H. Ross, A. S. Mahabaleshwarkar, and Y. Suhara, “When2Call: When (not) to Call Tools,” Proceedings of NAACL, 2025.
- [82] K. Ning, Y. Su, X. Lv, Y. Zhang, J. Liu, K. Liu, and J. Xu, “WTU-EVAL: A Whether-or-Not Tool Usage Evaluation Benchmark for Large Language Models,” arXiv:2407.12823, 2024.
- [83] J. Ye, G. Li, S. Gao, C. Huang, Y. Wu, S. Li, X. Fan, S. Dou, T. Ji, Q. Zhang, T. Gui, and X. Huang, “ToolEyes: Fine-Grained Evaluation for Tool Learning Capabilities of Large Language Models in Real-World Scenarios,” Proceedings of COLING, 2025, arXiv:2401.00741.
- [84] S. G. Patil, H. Mao, C.-J. Ji, F. Yan, V. Suresh, I. Stoica, and J. E. Gonzalez, “The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models,” Proceedings of ICML, 2025.
- [85] J. Wang, J. Zhou, X. Wang, X. Mo, H. Zhang, Q. Lin, C. Jin, M. Wen, W. Zhang, Q. Peng, and J. Wang, “HammerBench: Fine-Grained Function-Calling Evaluation in Real Mobile Assistant Scenarios,” Findings of ACL, 2025, arXiv:2412.16516.
- [86] K. Basu, I. Abdelaziz, K. Kate, M. Agarwal, M. Crouse, Y. Rizk, K. Bradford, A. Munawar, S. Kumaravel, S. Goyal, X. Wang, L. Lastras, and P. Kapanipathi, “NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls,” Proceedings of EMNLP, 2025, arXiv:2409.03797.
- [87] N. Farn and R. Shin, “ToolTalk: Evaluating Tool-Usage in a Conversational Setting,” arXiv: 2311.10775, 2023.
- [88] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, “ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities,” Findings of NAACL, 2025, arXiv:2408.04682.

- [89] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” arXiv:2406.12045, 2024.
- [90] C. Chen, X. Hao, W. Liu, X. Huang, X. Zeng, S. Yu, D. Li, S. Wang, W. Gan, Y. Huang, W. Liu, X. Wang, D. Lian, B. Yin, Y. Wang, and W. Liu, “ACEBench: Who Wins the Match Point in Tool Usage?,” arXiv: 2501.12851, 2025.
- [91] P. Yu, W. Liu, Y. Yang, J. Li, Z. Zhang, X. Feng, and F. Zhang, “Benchmarking LLM Tool-Use in the Wild,” arXiv:2604.06185, 2026.
- [92] F. Lei, Y. Yang, W. Sun, and D. Lin, “MCPVerse: An Expansive, Real-World Benchmark for Agentic Tool Use,” arXiv:2508.16260, 2025.
- [93] B. Elder, A. Murthi, J. Kang, A. R. Naik, K. Kate, K. Basu, and D. Contractor, “Live API-Bench: 2500+ Live APIs for Testing Multi-Step Tool Calling,” Proceedings of EACL, 2026.
- [94] J. Ye, Y. Wu, S. Gao, S. Li, G. Li, X. Fan, Q. Zhang, T. Gui, and X. Huang, “RoTBench: A Multi-Level Benchmark for Evaluating the Robustness of Large Language Models in Tool Learning,” Proceedings of EMNLP, 2024, arXiv:2401.08326.
- [95] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X. Huang, “ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages,” Proceedings of ACL, 2024, arXiv:2402.10753.
- [96] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the Risks of LM Agents with an LM-Emulated Sandbox,” Proceedings of ICLR, 2024, arXiv: 2309.15817.
- [97] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” arXiv: 2406.13352, 2024.
- [98] E. Rabinovich and A. Anaby Tavor, “On the Robustness of Agentic Function Calling,” Proceedings of TrustNLP Workshop at NAACL, 2025, arXiv:2504.00914.
- [99] Y.-C. Chen, P.-C. Hsu, C.-J. Hsu, and D.-S. Shiu, “Enhancing Function-Calling Capabilities in LLMs: Strategies for Prompt Formats, Data Integration, and Multilingual Translation,” Proceedings of NAACL Industry Track, 2025.
- [100] M. Chen, Y. Sun, K. Lu, et al., “Facilitating Multi-turn Function Calling for LLMs via Compositional Instruction Tuning,” arXiv:2410.12952, 2024.
- [101] W. Liu, X. Huang, X. Zeng, X. Hao, S. Yu, D. Li, S. Wang, W. Gan, Z. Liu, Y. Yu, et al., “ToolACE: Winning the Points of LLM Function Calling,” Proceedings of ICLR, 2025, arXiv:2409.00920.
- [102] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng, et al., “APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets,” arXiv:2406.18518, 2024.

- [103] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, K. Ren, D. Li, and D. Yang, “EASYTOOL: Enhancing LLM-Based Agents with Concise Tool Instruction,” *Proceedings of NAACL*, 2025.
- [104] B. Patel, D. Belli, A. Jalalirad, M. Arnold, A. Ermolov, and B. Major, “Dynamic Tool Dependency Retrieval for Efficient Function Calling,” *arXiv:2512.17052*, 2025.
- [105] Z. Zhang, et al., “ToolExpNet: Optimizing Multi-Tool Selection in LLMs with Similarity and Dependency-Aware Experience Networks,” *Findings of ACL*, 2025.
- [106] X. Fei, X. Zheng, and H. Feng, “MCP-Zero: Proactive Toolchain Construction for LLM Agents from Scratch,” *arXiv:2506.01056*, 2025.
- [107] X. Gao, S. Xie, J. Zhai, S. Ma, and C. Shen, “MCP-Radar: A Multi-Dimensional Benchmark for Evaluating Tool Use Capabilities in Large Language Models,” *arXiv:2505.16700*, 2025.
- [108] Q. Lin, M. Wen, Q. Peng, G. Nie, J. Liao, J. Wang, X. Mo, J. Zhou, C. Cheng, Y. Zhao, et al., “HAMMER: Robust Function-Calling for On-Device Language Models via Function Masking,” *arXiv:2410.04587*, 2024.
- [109] L. E. Erdogan, N. Lee, S. Jha, S. Kim, R. Tabrizi, S. Moon, C. Hooper, G. Anumanchipalli, K. Keutzer, and A. Gholami, “TinyAgent: Function Calling at the Edge,” *arXiv:2409.00608*, 2024.
- [110] W. Zhang, Y. Zhang, L. Zhu, Q. Jia, F. Jiang, H. Guo, Z. Li, and M. Zhou, “ADC: Enhancing Function Calling via Adversarial Datasets and Code Line-Level Feedback,” *arXiv:2412.17754*, 2024.
- [111] B. Hao, et al., “FunReason: Enhancing Large Language Models’ Function Calling for Complex Reasoning,” *arXiv:2505.20192*, 2025.
- [112] S. Gao, Z. Shi, M. Zhu, B. Fang, X. Xin, P. Ren, Z. Chen, J. Ma, and Z. Ren, “Confucius: Iterative Tool Learning from Introspection Feedback by Easy-to-Difficult Curriculum,” *Proceedings of AAAI*, 2024.
- [113] X. Liu, Z. Peng, X. Yi, X. Xie, L. Xiang, Y. Liu, and D. Xu, “ToolNet: Connecting Large Language Models with Massive Tools via Tool Graph,” *arXiv:2403.00839*, 2024.
- [114] Z. Liu, Z. Lai, Z. Gao, E. Cui, Z. Li, X. Zhu, L. Lu, Q. Chen, Y. Qiao, J. Dai, et al., “ControlLLM: Augment Language Models with Tools by Searching on Graphs,” *Proceedings of ECCV*, 2024.
- [115] N. Braunschweiler, R. Doddipatla, and T.-C. Zorila, “ToolReAGt: Tool Retrieval for LLM-Based Complex Task Solution via Retrieval Augmented Generation,” *Proceedings of KnowFM Workshop*, 2025.
- [116] L. Gao, Y. Wang, M. Peng, J. Tang, Y. Shang, M. Sun, and J. Su, “Tool Graph Retriever: Exploring Dependency Graph-Based Tool Retrieval for Large Language Models,” *arXiv:2508.05152*, 2025.
- [117] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong, “ReTool: Reinforcement Learning for Strategic Tool Use in LLMs,” *arXiv:2504.11536*, 2025.
- [118] Z. Li, et al., “Reinforcement Learning for LLM Agent Planning,” *Proceedings of EMNLP Industry Track*, 2025.

- [119] H. Song, J. Jiang, Y. Min, J. Chen, Z. Chen, W. Z. Zhao, L. Fang, and J.-R. Wen, “R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning,” arXiv:2503.05592, 2025.
- [120] Q. Zhang, S. Yang, L. Gao, H. Chen, X. Hu, J. Chen, J. Wang, S. Guo, B. Zheng, H. Wang, and J. Zhao, “LeTS: Learning to Think-and-Search via Process-and-Outcome Reward Hybridization,” Proceedings of EMNLP, 2025.
- [121] Y. Zheng, D. Fu, X. Hu, X. Cai, L. Ye, P. Lu, and P. Liu, “DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-World Environments,” arXiv:2504.03160, 2025.
- [122] H. Song, et al., “Asynchronous LLM Function Calling,” arXiv:2412.07017, 2024.
- [123] Y.-C. Chen, et al., “Can a Single Model Master Both Multi-turn Conversations and Tool Use?,” arXiv: 2502.08820, 2025.
- [124] J. Shi, et al., “Prompt Injection Attack to Tool Selection in LLM Agents,” arXiv:2504.19793, 2025; also Proceedings of NDSS, 2026.
- [125] T. Zhang, et al., “Imprompter: Tricking LLM Agents into Improper Tool Use,” arXiv:2410.14923, 2024.
- [126] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, et al., “PromptArmor: Simple yet Effective Prompt Injection Defenses,” arXiv:2507.15219, 2025.
- [127] T. Zhang, Y. Xu, J. Wang, K. Guo, X. Xu, B. Xiao, Q. Guan, J. Fan, J. Liu, Z. Liu, and H. Hu, “AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification,” arXiv:2602.22724, 2026.
- [128] M. A. Ferrag, et al., “From Prompt Injections to Protocol Exploits: Threats in LLM-Agent Ecosystems,” ICT Express, 2025.
- [129] E. Hubinger, C. Denison, J. Mu, M. Lambert, M. Tong, M. MacDiarmid, T. Lanham, D. Ziegler, N. Cheng, A. Jermyn, et al., “Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training,” arXiv: 2401.05566, 2024.
- [130] Z. Zhang, L. Lei, L. Wu, R. Sun, Y. Huang, C. Long, X. Liu, X. Lei, J. Tang, and M. Huang, “SafetyBench: Evaluating the Safety of Large Language Models,” Proceedings of ACL, 2024.
- [131] Z. Zhang, Y. Zhang, L. Li, H. Gao, L. Wang, H. Lu, F. Zhao, Y. Qiao, and J. Shao, “PsySafe: A Comprehensive Framework for Psychological-Based Attack, Defense, and Evaluation of Multi-Agent System Safety,” Proceedings of ACL, 2024.
- [132] A. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, “Augmenting Large Language Models with Chemistry Tools,” Nature Machine Intelligence, 2024.
- [133] Y. Zhuang, Y. Yu, K. Wang, H. Sun, and C. Zhang, “ToolQA: A Dataset for LLM Question Answering with External Tools,” NeurIPS Datasets and Benchmarks Track, 2023, arXiv:2306.13304.
- [134] C. Ma, G. Li, K. C. C. Chang, F. Huang, R. Cheng, and Y. Li, “Can LLM Already Serve as A Database Interface? A Blg Bench for Large-Scale Database Grounded Text-to-SQLs,” arXiv:2305.03111, 2023.

- [135] M. Tian, L. Gao, S. D. Zhang, X. Chen, C. Fan, X. Guo, R. Haas, P. Ji, K. Krongchon, Y. Li, et al., “SciCode: A Research Coding Benchmark Curated by Scientists,” *Proceedings of NeurIPS*, 2024.
- [136] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, et al., “BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions,” *arXiv:2406.15877*, 2024.
- [137] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. Le, and T. Luong, “FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation,” *Findings of ACL*, 2024.
- [138] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen, “Tool Learning with Large Language Models: A Survey,” *Frontiers of Computer Science*, 2025, *arXiv:2405.17935*.
- [139] Z. Shen, “LLM With Tools: A Survey,” *arXiv:2409.18807*, 2024.
- [140] J. Hu, M. Zhong, K. Chen, X. Bai, and M. Zhang, “Agentic Tool Use in Large Language Models,” *arXiv:2604.00835*, 2026.
- [141] X. Li, W. Jiao, J. Jin, G. Dong, J. Jin, Y. Wang, H. Wang, Y. Zhu, J.-R. Wen, Y. Lu, and Z. Dou, “A Review of Prominent Paradigms for LLM-Based Agents,” *Proceedings of COLING*, 2025.
- [142] J. Chen, H. Wu, J. Pang, Y. Wang, D. Zhang, and C. Sun, “Tool Learning with Language Models: A Comprehensive Survey of Methods, Pipelines, and Benchmarks,” 2025.
- [143] W. Xu, et al., “LLM-Based Agents for Tool Learning: A Survey,” 2025.
- [144] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., “A Survey on Large Language Model Based Autonomous Agents,” *Frontiers of Computer Science*, 2024.
- [145] L. Li, et al., “A Survey on Evaluation of LLM-Based Agents,” *arXiv:2503.16416*, 2025.
- [146] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, J. Dwivedi-Yu, A. Celikyilmaz, E. Grave, Y. LeCun, and T. Scialom, “Augmented Language Models: A Survey,” *Transactions on Machine Learning Research*, 2023, *arXiv:2302.07842*.
- [147] P. Zhao, Z. Jin, and N. Cheng, “An In-Depth Survey of Large Language Model-Based Artificial Intelligence Agents,” *arXiv:2309.14365*, 2023.
- [148] V. Cohen, S. Huang, O. Shodhan, and Y. Artzi, “A Survey of Robotic Language Grounding,” *Proceedings of IJCAI*, 2024, *arXiv:2405.13245*.

1 7 23 <https://arxiv.org/abs/2405.17935>
<https://arxiv.org/abs/2405.17935>

2 4 8 <https://arxiv.org/abs/2303.09014>
<https://arxiv.org/abs/2303.09014>

- 3 <https://arxiv.org/abs/1207.1420>
<https://arxiv.org/abs/1207.1420>
- 5 <https://arxiv.org/abs/2305.16291>
<https://arxiv.org/abs/2305.16291>
- 6 [2307.16789] ToolLLM: Facilitating Large Language Models to ...
https://arxiv.org/html/2307.16789?utm_source=chatgpt.com
- 9 <https://arxiv.org/abs/2210.02875>
<https://arxiv.org/abs/2210.02875>
- 10 Language Models Can Teach Themselves to Use Tools
https://www.semanticscholar.org/paper/Toolformer%3A-Language-Models-Can-Teach-Themselves-to-Schick-Dwivedi-Yu/53d128ea815bcc0526856eb5a9c42cc977cb36a7?utm_source=chatgpt.com
- 11 <https://arxiv.org/abs/2308.00675>
<https://arxiv.org/abs/2308.00675>
- 12 <https://arxiv.org/abs/2410.12952>
<https://arxiv.org/abs/2410.12952>
- 13 14 <https://openreview.net/forum?id=2GmDdhBdDk>
<https://openreview.net/forum?id=2GmDdhBdDk>
- 15 <https://aclanthology.org/2025.emnlp-industry.116.pdf>
<https://aclanthology.org/2025.emnlp-industry.116.pdf>
- 16 <https://arxiv.org/pdf/2410.04587>
<https://arxiv.org/pdf/2410.04587>
- 17 <https://arxiv.org/abs/2207.05608>
<https://arxiv.org/abs/2207.05608>
- 18 20 <https://aclanthology.org/2025.naacl-long.174/>
<https://aclanthology.org/2025.naacl-long.174/>
- 19 <https://aclanthology.org/2025.acl-long.1481.pdf>
<https://aclanthology.org/2025.acl-long.1481.pdf>
- 21 <https://arxiv.org/abs/2402.10753>
<https://arxiv.org/abs/2402.10753>
- 22 <https://arxiv.org/abs/2604.06185>
<https://arxiv.org/abs/2604.06185>